

An Ontology Design Pattern for Software Requirements

Valentina Anita Carriero^{1,*}, Marco Grassi¹, Alessio Carenini¹ and Irene Celino¹

¹Cefriel, viale Sarca 226, 20126 Milano, Italy

Abstract

Software often depends on specific software and hardware resources to be installed, deployed, and executed correctly. However, existing ontologies about software only partially support the representation of such requirements, which are often expressed as free text or through direct properties. This paper presents an Ontology Design Pattern for representing software requirements as first-class entities. The pattern distinguishes between software and hardware requirements, supports requirements over specific resources or resource types, and allows one to express exact, minimum, and maximum required values for properties and software versions. We also discuss how the pattern can be complemented with SHACL shapes to represent and validate more complex requirements involving nested components or alternatives. The approach is illustrated through a usage example inspired by the SMARTY project, where software requirements are matched against descriptions of available software and hardware resources in a catalogue.

Keywords

software requirements, ontology, knowledge engineering, ontology design patterns, SHACL

1. Introduction

To function, software often depends on specific software and hardware resources to be installed, deployed, and executed correctly. These requirements may concern, for example, operating systems, software dependencies, hardware types, processor architectures, memory, storage, or other measurable properties. Although such information is commonly provided in documentation, configuration files, or catalogues, it is often represented as free text or through ad hoc fields, limiting interoperability and automated processing.

Existing ontologies on software mainly focus on metadata, licensing, provenance, releases, or domain-specific software classification. Some of them include properties for representing software requirements, but these requirements are usually not modelled as first-class entities and provide limited support for expressing property-specific constraints, version intervals, nested component requirements, or alternatives.

This paper proposes an Ontology Design Pattern (ODP) for representing software requirements as explicit resources. In this work, the term requirement refers to system requirements, and should not be confused with functional and non-functional requirements typically collected during software engineering processes to specify the expected behaviour and quality attributes of a software system. The pattern distinguishes between software and hardware requirements, supports requirements expressed over specific resources or resource types, and allows one to represent exact, minimum, and maximum required values. Since some requirements show a higher level of complexity, and are better understood as constraints to be checked over concrete descriptions of available resources, the paper also discusses how the ODP can be complemented with SHACL shapes. This combination keeps the OWL pattern simple and easily reusable, while enabling the validation of more complex requirements in catalogue-based scenarios.

Seventeenth Workshop on Ontology Design and Patterns (WOP 2026), co-located with ISWC 2026: 25th International Semantic Web Conference, October 26, 2026, Bari, Italy

*Corresponding author.

✉ valentina.carriero@cefriel.com (V. A. Carriero); marco.grassi@cefriel.com (M. Grassi); alessio.carenini@cefriel.com (A. Carenini); irene.celino@cefriel.it (I. Celino)

ORCID 0000-0003-1427-3723 (V. A. Carriero); 0000-0003-3139-3049 (M. Grassi); 0000-0003-1948-807X (A. Carenini); 0000-0001-9962-7193 (I. Celino)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

The proposed pattern is illustrated through a usage example inspired by the SMARTY project, where a matchmaker compares the requirements of a software component with the characteristics of available software and hardware resources.

2. Related Work

Many ontologies have been developed for representing software and related metadata.

The Software Description (SD) Ontology¹ [1] provides a model for describing software components, covering both metadata (e.g., attribution, licensing, and usage information) and technical aspects such as inputs, outputs, and variables. It has been used for the OKG-Soft knowledge graph [1], which includes machine readable scientific software metadata. The SD ontology builds upon Schema.org² and Codemeta³ vocabularies and further extends OntoSoft⁴ [2], which is the ontology adopted by the OntoSoft portal⁵ for software metadata management. Codemeta itself provides a comprehensive set of software attribution properties, including authorship, maintenance, funding sources, licensing information, and links to related publications.

As for modelling requirements, Schema.org includes the following relevant properties: `softwareRequirements` (requirements needed to install/run a software component, like Java), `(minimum) memoryRequirements`, `supported operatingSystem(s)` (the OS the software can run on, e.g., Windows 7), `processorRequirements`, that is, the processor architecture required to run the application (e.g. x86), `runtimePlatform` or script interpreter dependencies (for instance, Java 25, Python 3.0), `storageRequirements` (free space required). The first four properties have also been reused (`subPropertyOf`) in the Software Description Ontology, as datatype properties, thus, such ontologies partially cover the representation of requirements. However, these are represented as free text in SD, while in schema.org they expect either an instance of a class (e.g., `OperatingSystem`) or a text/url, lacking some expressiveness and not supporting the representation of more complex requirements.

OntoSoft includes `requiresAverageMemory` (with `MeasurementEntity` as range), `hasDependency` (with `Software` as range, answering to the question *What other software does the software require to be installed?*), and `supportsOperatingSystem` (with `OperatingSystem` as range). Even if these requirements point to instances of classes that can be further described, they are limited to average memory, dependencies and operating systems, and do not consider minimum/maximum versions, alternative requirements, etc.

The Core Software Ontology⁶ captures the essential concepts needed to represent both software systems and web services, including software entities, data, users, interfaces, access permissions, and related elements. Its design relies on the foundational DOLCE ontology⁷ and the ontology design patterns derived from it. This ontology models the concept of constraints, which can be applied to computational tasks, not directly to software, “e.g., a Web service policy might state that an invocation is only possible with Kerberos or X509 authentication”. Instead, the datatype property `method-requires` is related to `classes` as a special kind of software.

The Software Ontology⁸ (SWO) [3] models software tools together with their categories, functionalities, versions, licenses, provenance, and associated tasks. Developed within the OBO Foundry (Open Biological and Biomedical Ontology Foundry) ecosystem, SWO was created to support interoperability in the biological and biomedical domains. It contains rich descriptions of software applications, formats, and licensing aspects, with a particular focus on bioinformatics software.

¹sd: <https://w3id.org/okn/o/sd#>

²<https://schema.org/>

³<https://codemeta.github.io/terms/>

⁴<https://ontosoft-earthcube.github.io/ontosoft/ontosoft%20ontology/v1.0.1/doc/index.html>

⁵<https://www.ontosoft.org/portal/#list>

⁶<https://km.aifb.kit.edu/sites/cos/>

⁷<https://www.loa.istc.cnr.it/dolce/overview.html>

⁸<https://github.com/allysonlister/swo>

Table 1

Competency questions answered by the Software Requirements ODP.

ID	Competency question
CQ1	Which is the (software/hardware) requirement of a software?
CQ2	Which is the software/hardware required by a software?
CQ3	Which is the type of software/hardware required by a software?
CQ4	Which is the required (minimum/maximum/exact) value of a property of a software/hardware required by a software?
CQ5	Which is the required (maximum/minimum/exact) version of a software required by a software?

The DevOps Infrastructure Ontology Network (DevOps-Infra)⁹ aims at representing the main sets of entities and relationships used in the context of DevOps infrastructure. It includes 11 ontology modules, two of which dedicated to software¹⁰ and hardware¹¹. The Software module models 7 classes and 13 properties, addressing software versions, releases, deploy/install packages.

Neither SWO nor DevOps-Infra addresses software requirements.

The SPDX ontology¹² represents the semantic version of the SPDX standard, which is widely used for exchanging Software Bill of Materials (SBOM) information, including software components, licenses, copyright statements, and security-related references. SPDX provides a general model for representing relationships between SPDX elements. Its vocabulary of relationship types includes `dependsOn`, `hasPrerequisite`, and `hasRequirement`, which can be used to state that an SPDX element depends on, has a prerequisite on, or has a requirement on another element, however, such relationship between elements is not modelled further.

Taken together, the literature confirms that software requirements are usually not treated as first-class entities in existing ontologies, nor they do allow for a strong expressiveness (e.g., property-specific minimum/maximum values, software requirements involving alternative options, etc.).

3. The Software Requirements ODP

A diagram depicting the pattern is given in Figure 1¹³, and the Competency Questions (CQs) addressed by our pattern are included in Table 1. The competency questions are intentionally organized at different levels of granularity. CQ1 focuses on identifying the requirements associated with a software, while CQ2 and CQ3 distinguish whether the requirement refers to a specific software/hardware resource or to a resource type. CQ4 and CQ5 further refine the description of requirements by addressing (quantitative) constraints on resource properties and software versions, respectively. The ODP has been submitted to the ODPa Ontology Design Patterns Repository¹⁴.

Requirement. Requirements are represented as individuals of the class `:Requirement`. A requirement is intended as an explicit description of a condition that a software component, or the environment in which it is executed, has to satisfy. Requirements must be satisfied by the execution environment in order for a software to be installed, deployed, and executed. Such system requirements are different from the functional and non-functional requirements typically elicited during software engineering processes, which are not covered by our pattern. The pattern distinguishes between requirements concerning software resources and requirements concerning hardware resources, through the classes `:SoftwareRequirement` and `:HardwareRequirement`, which are declared as subclasses of `:Requirement`. A software resource is associated with its requirements by means of the object property `:hasRequirement`. This allows one to describe, for instance, that a given software requires another

software, a specific type of hardware, or a value for one of the properties of the required resource.

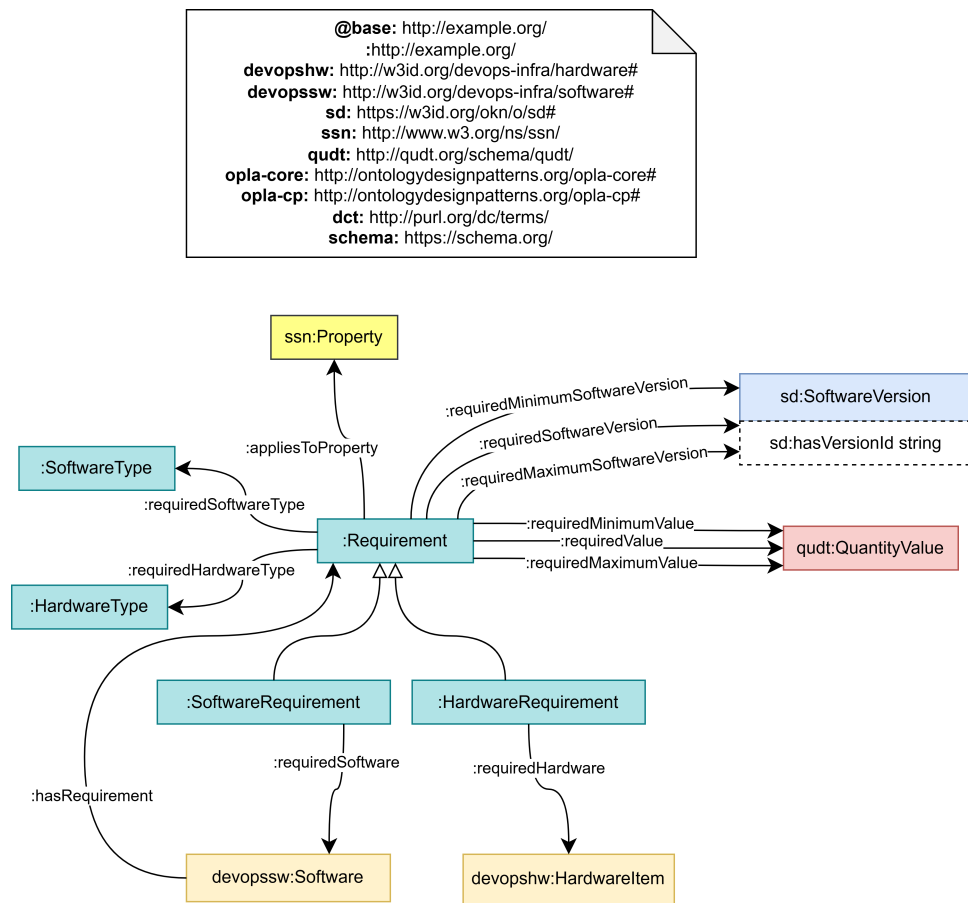


Figure 1: Pattern for Software Requirements

SoftwareRequirement and HardwareRequirement. Software requirements and hardware requirements are used to specify the kind of resource that is needed by a software in order for it to be executed. In the case of software requirements, the object property `:requiredSoftware` relates a `:SoftwareRequirement` to a software resource that is required, modelled as an instance of `devopssw:Software`. In the case of hardware requirements, the object property `:requiredHardware` relates a `:HardwareRequirement` to the required `devopshw:HardwareItem`. The pattern also supports requirements expressed at the level of resource types, through the properties `:requiredSoftwareType` and `:requiredHardwareType`. These properties respectively relate a requirement to an instance of `:SoftwareType` or `:HardwareType`, representing a category of software or hardware resources that may satisfy the requirement. They are used when the requirement does not identify a specific resource but a category of acceptable resources, such as *a GPU*. The distinction between resources and resource types therefore allows requirements to refer either to a specific software or hardware item or to any resource belonging to the required category. The classes `:HardwareType` and `:SoftwareType` can be populated by the ODP adopters with the instances needed for their application domain.

appliesToProperty and required values. Requirements may constrain a specific property of the required resource. For example, a software may require a CPU whose memory is above a given threshold. The object property `:appliesToProperty` is used to associate a requirement with the property that is

⁹<https://oeg-upm.github.io/devops-infra/index.html>

¹⁰`devopssw:` <http://w3id.org/devops-infra/software#>

¹¹`devopshw:` <http://w3id.org/devops-infra/hardware#>

¹²<http://spdx.org/rdf/terms>

¹³The diagram follows the Chowlk notation <https://chowlk.linkeddata.es/>

¹⁴<https://github.com/odpa/patterns-repository/tree/master/SoftwareRequirements>

constrained by the requirement. The constrained property is represented as a `ssn:Property`¹⁵. The value of such property, as imposed by a requirement, is represented through the properties `:requiredValue`, `:requiredMinimumValue`, and `:requiredMaximumValue`. These properties allow one to distinguish between exact value requirements, lower-bound requirements, and upper-bound requirements. The required value is modelled as a `qudt:QuantityValue`¹⁶, so that quantitative values can be represented together with their `qudt:Units` of measure when applicable. For instance, a software requirement may state that the required RAM must have a size of at least 32 GB (`:requiredMinimumValue`).

SoftwareVersion. Requirements concerning software versions are represented separately from quantitative value requirements. Software versions are modelled as instances of `sd:SoftwareVersion`, since version identifiers are not always purely numerical values and may include strings such as Ubuntu 26.04 LTS. The properties `:requiredSoftwareVersion`, `:requiredMinimumSoftwareVersion`, and `:requiredMaximumSoftwareVersion` are used to express exact, minimum, and maximum version requirements, respectively. The actual version identifier is represented through `sd:hasVersionId`.

3.1. Extending the pattern with SHACL for complex requirements

While the ODP introduced above is adequate to represent structurally simple software requirements, e.g., when they identify a required software or hardware resource, a required type, or a single constrained property, which are to be used for documentation purposes, OWL alone is less suitable when requirements become more complex and nested, and must be interpreted as constraints to be checked against concrete deployment descriptions [4, 5]. OWL was designed to support inference over potentially incomplete knowledge graphs under the open-world assumption, whereas requirement checking typically assumes that the available description of the execution environment is complete for the purpose of validation. A statement such as “software *s* requires a GPU” can be represented in OWL by linking a requirement instance to a hardware type, but requirements such as “software *s* requires a software *s1* of model *m*, having a GPU with at least *n* GB of memory and feature *f*” can be represented in greater detail and better expressiveness with SHACL. Similar motivations have been discussed in the context of regulatory requirements, often containing a number of alternative requirements, and that have to be verified over instance data [5]. [5] demonstrates how complex regulations can be easily expressed as SHACL constraints. Such limitations have been reported also in other application domains, where OWL was found adequate for domain conceptualization and interoperability, but not for enforcing detailed information requirements, conditional dependencies, or data completeness conditions [4].

More generally, the literature has explored practical applications of SHACL in applicative scenarios [6], and consistently characterizes OWL and SHACL as complementary rather than competing technologies [5]. This combined OWL+SHACL approach has been recommended in recent work on ontology engineering, where classes and properties are maintained in OWL for domain modeling, and shapes are introduced to express data-quality and conformance requirements over the corresponding knowledge graphs [4]. Recent work has further shown that shapes and constraints can themselves be developed in a pattern-oriented way, extracted from recurring modeling practices, or generated semi-automatically from ontologies and data, although manual refinement remains necessary [7, 8, 9].

Based on the literature, we conclude that SHACL is particularly appropriate when requirements involve nested component descriptions, explicit alternatives, or disjunctive conditions. In this perspective, the Software Requirements ODP can be complemented with a SHACL layer that formalizes those aspects of the requirements [4], thus using SHACL when requirement complexity exceeds what can be modeled clearly and maintainably in OWL alone. Concretely, each requirement individual already represented in the ODP can be associated with one or more SHACL node shapes targeting the corresponding software or hardware descriptions, thus making explicit which triples must be present and which values are acceptable. Formally, the requirement and the corresponding SHACL node shape are connected through the required resource: the requirement identifies the resource through `:requiredSoftware` or

¹⁵ssn: <http://www.w3.org/ns/ssn/>

¹⁶Quantities, Units, Dimensions and Types (QUDT) ontology, qudt: <http://qudt.org/schema/qudt/>

:requiredHardware, while the shape targets the same resource through `sh:targetNode`, as illustrated in Section 4.

4. Usage Example

This section illustrates how the Software Requirements ODP can be applied to represent software requirements at different levels of complexity. In Figure 2, we present an example including *simple* requirements that do not require SHACL to be exhaustively expressed, while in the SHACL snippets we show more complex requirements represented as shapes.

This example is motivated by a concrete use case from the SMARTY research project¹⁷, which focuses on scalable and quantum-resilient heterogeneous edge computing enabling trustworthy AI, and includes a specific activity that addresses the semantic management of digital assets (including software) for supporting interoperability. In the context of one of the use cases of the project, a matchmaker is being developed, which addresses the dynamic allocation of hardware resources (DPU and GPU) based on the characteristics and requirements of incoming service requests, by comparing the requirements expressed for a given software with the actual characteristics of software and hardware resources described in a catalogue. The goal of the matchmaker is to identify candidate resources that can be used to execute, support, or deploy the main software.

The instance `data:network-analysis-software`, of class `devopssw:Software`, is associated with different requirements through `:hasRequirement`. The requirement `data:requires-networking-hardware` states that the software requires a hardware resource of type `data:networking-hardware (:requiredHardwareType)`. This allows the required resource to be specified at the type level, without referring to a specific hardware item.

A more specific hardware requirement is represented by `data:requires-interface-card`. Such requirement refers to the hardware type `data:network-interface-card` and constrains one of its properties, namely `data:max-throughput`, through `:appliesToProperty`. The minimum required value is represented with a `qudt:QuantityValue`, having 10 as numeric value and `unit:GigaBIT-PER-SEC` as measurement unit. This means that the network analysis software requires a network interface card supporting at least 10 Gbps.

The requirement `data:requires-ubuntu` represents a software dependency. It links the network analysis software to the operating system `data:Ubuntu`, an instance of `devopssw:Software`, through `:requiredSoftware`. The same requirement also specifies the accepted version interval: the minimum version is `data:Ubuntu-24.04.LTS`, whose identifier is "24.04 LTS" (`sd:hasVersionId`), while the maximum version is `data:Ubuntu-25.10`. Both are modelled as instances of `sd:SoftwareVersion`. The software `data:Ubuntu` is also connected to these version instances through `sd:hasSoftwareVersion`. Therefore, the example represents that the network analysis software requires Ubuntu, with a version ranging from Ubuntu 24.04 LTS to Ubuntu 25.10.

The example requirements presented above are intentionally kept simple: requirements are represented as first-class resources, associated with the software that declares them, and specify the required software or hardware resource, the property involved in the requirement, and the required value, when this value can be represented in a direct way. This is sufficient for requirements such as “the software requires Ubuntu between version 24.04 LTS and 25.10”, or “the software requires a network interface card supporting at least 10 Gbps throughput”. In these cases, the requirement can be represented through a small number of RDF triples, without introducing complex OWL expressions.

However, in the SMARTY project, some requirements involve several connected components. For example, a network analysis software may require a DPU that includes a CPU having specific characteristics and related components. Other requirements include alternatives, such as the case where a software requires a CPU with either one architecture or another. Although some of these aspects could be partially represented in OWL, this would require complex class expressions and would make the pattern less reusable and harder to understand [10]. For this reason, we use OWL and SHACL for

¹⁷<https://www.smarty-project.eu/>

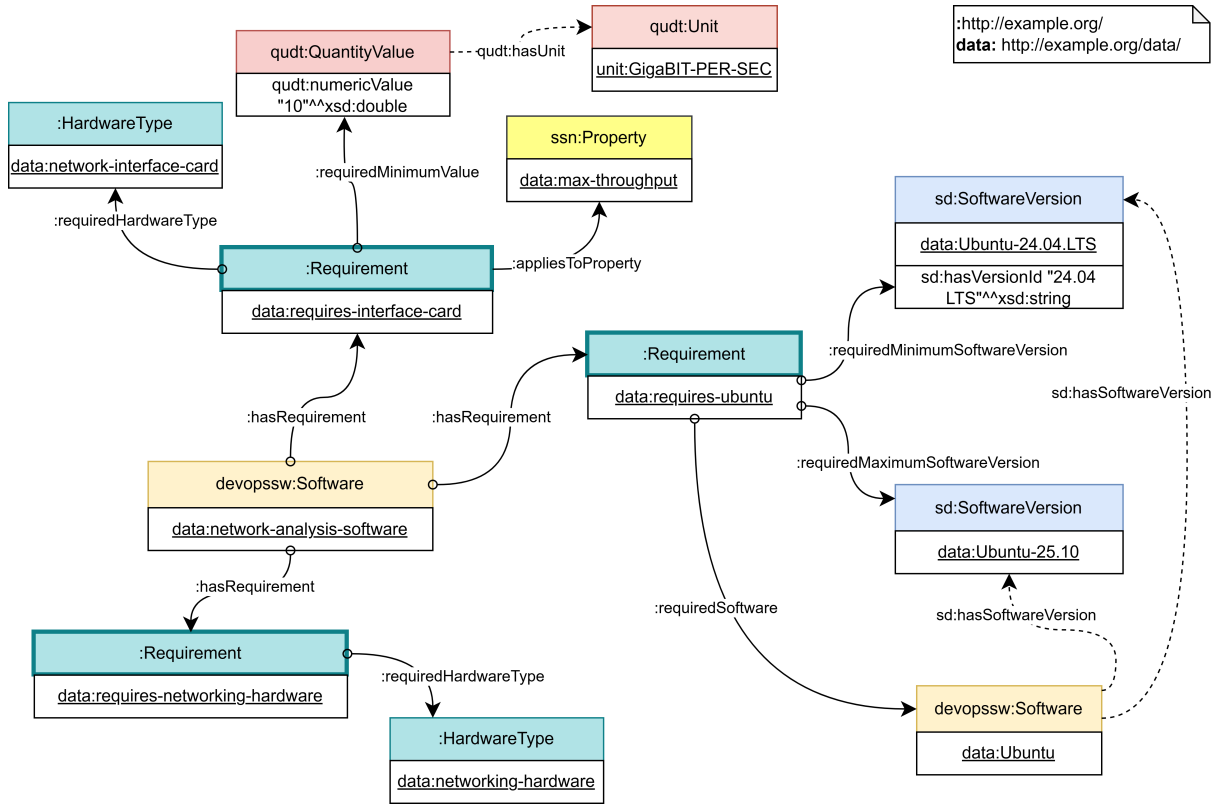


Figure 2: Usage example of the Software Requirements ODP

different, complementary purposes. SHACL provides a straightforward way to turn complex software requirements into executable constraints, while preserving the simplicity and reusability of the OWL pattern.

This modelling choice is particularly useful in the catalogue-based scenario of SMARTY. A requirement node can state that a software requires a certain kind of hardware or software resource, while a SHACL shape can specify what a matching catalogue resource must look like. The shape can check exact values, minimum or maximum values, required features, nested components, and alternatives. Therefore, the OWL pattern is not replaced by SHACL. Rather, OWL provides the reusable vocabulary for representing requirements, while SHACL provides the validation layer used by the matchmaker to compare them with the actual descriptions of available resources.

For instance, the following example shows a possible SHACL extension of the Software Requirements pattern for expressing a nested component. The requirement states that the software needs a DPU with certain characteristics. The detailed characteristics that a candidate DPU must satisfy are represented in a SHACL node shape. The shape can then be used by the matchmaker to validate the DPU resources described in the catalogue and to select the ones that conform to the requirement.

In the example, for the example domain classes/properties (e.g., *has hardware model*) we also use the `data:` prefix. The software `data:network-analysis-software` declares the requirement `data:requires-dpu`. `data:requires-DPU` is an instance of `:Requirement`, and has the DPU `data:DPU-required-by-network-analysis-software` as `:requiredHardware`. The node shape `data:DPU-required-by-network-analysis-software-shape` is linked to the DPU it targets with `sh:targetNode`. Such shape specifies that the DPU must have `data:NVIDIA_BLUEFIELD` as hardware model, and must have a GPU that provides `data:CudaSupport` as feature.

```
@prefix : <http://example.org/> .
@prefix data: <http://example.org/data/> .
@prefix devopssw: <http://w3id.org/devops-infra/software> .
```

```

@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#>

data:network-analysis-software a devopssw:Software ;
    :hasRequirement data:requires-DPU .

data:requires-DPU a :Requirement ;
    :requiredHardware data:DPU-required-by-network-analysis-software .

data:DPU-required-by-network-analysis-software a data:DPU .

data:DPU-required-by-network-analysis-software-shape a sh:NodeShape ;
    sh:targetNode data:DPU-required-by-network-analysis-software ;
    sh:property [
        sh:path data:hasHardwareModel ;
        sh:hasValue data:NVIDIA_BLUEFIELD ;
        sh:minCount 1 ;
        sh:message "The candidate DPU
                    must have hardware model NVIDIA BlueField." ;
    ] ;

    sh:property [
        sh:path data:hasGPU ;
        sh:minCount 1 ;
        sh:node [
            sh:property [
                sh:path data:providesFeature ;
                sh:hasValue data:CudaSupport ;
                sh:message "The GPU of the candidate DPU
                            must provide CUDA support." ;
            ] ;
            sh:message "The candidate DPU
                        must have a GPU providing the required feature." ;
        ] .
    ] .

```

The matchmaker can apply `data:DPU-required-by-network-analysis-software-shape` to the DPU resources contained in the catalogue. Such shape could be translated into the following SPARQL query in order to find DPUs that satisfy such requirements in the catalogue.

```

PREFIX : <http://example.org/>
PREFIX data: <http://example.org/data/>
PREFIX sh: <http://www.w3.org/ns/shacl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT DISTINCT ?candidatedPU
WHERE {
    ?candidatedPU a data:DPU ;
    data:hasHardwareModel data:NVIDIA_BLUEFIELD ;
    data:hasGPU ?gpu .

    ?gpu data:providesFeature data:CudaSupport .
}

```

The same approach can be used for requirements containing alternatives. For example, the same software `data:network-analysis-software` may require a CPU that has either ARMv8 or x86-64 as CPU architecture. Such requirement can be expressed directly with `sh:or`.

```
data:network-analysis-software :hasRequirement data:requires-CPU .

data:requires-CPU a :Requirement ;
    :requiredHardware data:CPU-required-by-network-analysis-software .

data:CPU-required-by-network-analysis-software a data:CPU .

data:CPU-required-by-network-analysis-software-shape
    a sh:NodeShape ;
    sh:targetNode data:CPU-required-by-network-analysis-software ;

    sh:or (
        [
            sh:property [
                sh:path data:hasCPUArchitecture ;
                sh:hasValue data:ARMv8 ;
            ]
        ]
        [
            sh:property [
                sh:path data:hasCPUArchitecture ;
                sh:hasValue data:X86_64 ;
            ]
        ]
    ) ;

    sh:message "The candidate CPU
        must have either ARMv8 or x86-64 architecture." .
```

5. Conclusions

This paper presented an Ontology Design Pattern for representing software requirements as first-class entities. The pattern provides a lightweight model to describe software and hardware requirements, required resources or resource types, constrained properties, quantitative values, and software version intervals. In this way, it addresses a modelling need that is only partially covered by existing software description ontologies.

The paper also discussed the role of SHACL as a complementary layer for requirements that involve nested components, alternatives, or validation over concrete catalogue resources. In the proposed approach, OWL is used to provide the reusable conceptual structure of the pattern, while SHACL is used to operationalize complex requirements as executable constraints.

The SMARTY-inspired example shows how the pattern can support catalogue-based matchmaking, where software requirements need to be compared with the actual characteristics of available resources.

Future work will focus on further validating the pattern in real matchmaking scenarios, and evaluating its reuse across different domains where software deployment depends on heterogeneous hardware and software configurations.

Acknowledgements

This work is partially supported by the SMARTY project, co-funded by the European Commission (Grant id 101140087).

Declaration on Generative AI

During the preparation of this work, the authors used Microsoft 365 Copilot to paraphrase and reword. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

References

- [1] D. Garijo, M. Osorio, D. Khider, V. Ratnakar, Y. Gil, Okg-soft: An open knowledge graph with machine readable scientific software metadata, in: 2019 15th International Conference on eScience (eScience), IEEE, 2019, pp. 349–358.
- [2] Y. Gil, V. Ratnakar, D. Garijo, Ontosoft: Capturing scientific software metadata, in: Proceedings of the 8th International Conference on Knowledge Capture, 2015, pp. 1–4.
- [3] J. Malone, A. Brown, A. L. Lister, J. Ison, D. Hull, H. Parkinson, R. Stevens, The software ontology (swo): a resource for reproducibility in biomedical data analysis, curation and digital preservation, *Journal of biomedical semantics* 5 (2014) 25.
- [4] E. Ruckhaus, J. Toledo, E. A. Martínez Sarmiento, O. Corcho, Lessons learned from the combined development of owl and shacl, in: Proceedings of the 13th Knowledge Capture Conference 2025, 2025, pp. 1–4.
- [5] V. Heimsbakk, K. Torkelsen, Using the shapes constraint language for modelling regulatory requirements, *arXiv*, 2023.
- [6] L. Robaldo, S. Batsakis, On the interplay between validation and inference in shapes constraint language—an investigation on the time ontology, *Semantic Web* 17 (2026) 22104968261440710.
- [7] M. Poveda-Villalón, S. Carulli-Pérez, R. García-Castro, How much owl do you need to know to make sense of building ontologies?, *LDAC 2024* (2024) 12th.
- [8] V. A. Carriero, P. Groth, V. Presutti, Empirical ontology design patterns and shapes from wikidata, *Semantic Web* 15 (2024) 2293–2317.
- [9] L. Asprino, V. A. Carriero, V. Presutti, Extraction of common conceptual components from multiple ontologies, in: Proceedings of the 11th Knowledge Capture Conference, 2021, pp. 185–192.
- [10] A. Eberhart, C. Shimizu, S. Chowdhury, M. K. Sarker, P. Hitzler, Expressibility of owl axioms with patterns, in: European Semantic Web Conference, Springer, 2021, pp. 230–245.